Final Project in Numerical Programming

Student: Giorgi Gogsadze
Course: Numerical Programming
Professor: Ramaz Botchorishvili
Date: 27.01.2026

Dynamic Tracking and Shape Preservation

1. Problem Description

The objective of this project is to make the drone swarm:

1. transition from the static "Happy New Year!" greeting (end of previous problem) to an object detected in a video provided by me, and then
2. dynamically repeat the object's motion while maintaining shape preservation (the swarm moves like a rigid formation rather than deforming chaotically).

This sub-problem corresponds to the IVP with Velocity Tracking (IVP-VT) model described in the assignment.

Overall pipeline

The swarm first morphs from the static greeting into the object shape extracted from the first video frame using edge detection. Once initialized, dynamic motion is driven by dense optical flow, which is reduced to a rigid-body velocity estimate (translation + rotation). This rigid velocity is then tracked using an IVP-VT controller and integrated numerically using RK4.

2. Mathematical Model

2.1 Inputs, Initial conditions and parameters

State Variables

For each drone $\mathrm{i} = 1, \dots, \mathrm{N}$ $\mathrm{x_i(t)} \in \mathrm{R}^2$, $\mathrm{v_i(t)} \in \mathrm{R}^2$

The full state: $\mathrm{y(t)} = [\mathrm{x_1}, \dots, \mathrm{x_N}, \mathrm{v_1}, \dots, \mathrm{v_N}]$

Initial Conditions

The initial conditions for sub-problem 3 are inherited from the final state of sub-problem 2:

$$x_i(0) = x_i^{\mathrm{P}}, \quad v_i(0) = 0$$

Before tracking begins, drones are smoothly transitioned to match the object shape in the first video frame, obtained via edge detection and contour sampling.

2.2 Velocity Saturation

To prevent physically impossible motion and stabilize the integration, velocity is saturated:

$$\mathrm{sat}(\mathrm{v}) = \mathrm{v} \cdot \min\left(1, \frac{\mathrm{v_{max}}}{||\mathrm{v}||}\right).$$

In code this is applied to the kinematic term so that large accelerations do not instantly produce unrealistically large displacement per step.

2.3 IVP-VT Governing Equations (Velocity Field Tracking)

In Sub-problem 3 the target is not a static point $\mathrm{T_i}$. Instead, the swarm tries to match a time-varying velocity field extracted from the video.

The governing equations follow the assignment's **IVP VT** structure:

Position update

$$\dot{\mathrm{x}}_\mathrm{i} = \mathrm{sat}(\mathrm{v_i})$$

Velocity update

$$\dot{\mathrm{v}}_\mathrm{i} = \frac{1}{\mathrm{m}}\left[\mathrm{k_v}(\mathrm{V_{sat}}(\mathrm{x_i}, \mathrm{t}) - \mathrm{v_i}) + \sum_{\mathrm{j}\neq\mathrm{i}} \mathrm{f_{rep}}(\mathrm{x_i}, \mathrm{x_j}) - \mathrm{k_d v_i}\right].$$

Where:

- $\mathrm{V}(\mathrm{x}, \mathrm{t})$ is the optical-flow-based velocity field from the video,
- $\mathrm{V_{sat}}$ is the saturated version,
- $\mathrm{k_v}$ is a velocity tracking gain,
- $\mathrm{f_{rep}}$ prevents collisions,
- $\mathrm{k_d}$ damps oscillations.

2.4 Repulsion and Collision Avoidance

Collision avoidance is enforced by a short-range repulsion force:

$$f_{rep}(x_i, x_j) = \begin{cases} k_{rep}\dfrac{x_i - x_j}{\|x_i - x_j\|^3}, & \|x_i - x_j\| < R_{safe} \\ 0, & otherwise \end{cases}$$

This is intentionally strong at short distances (inverse-cube) and zero outside the safety radius, giving stable, collision-free behavior even when the tracked object changes direction quickly.

3. Edge Detection

At the beginning of Sub-problem 3, the drone swarm must transition from a static greeting formation to a spatial configuration matching the shape of a moving object in a video. To achieve this, the first video frame is processed to extract the object's visible structure.

The following steps are applied:

1. **Grayscale conversion**
   The first video frame is converted to grayscale to simplify intensity analysis.
2. **Canny edge detection**
   The Canny edge detector is applied:

$$E = \mathrm{Canny}\left(I, T_{\mathrm{low}}, T_{\mathrm{high}}\right)$$

where I is the grayscale image and $T_{\mathrm{low}}, T_{\mathrm{high}}$ are empirically selected thresholds.

3. **Contour extraction**
   Connected edge pixels are grouped into contours, which provide ordered point sets describing the object boundary.
4. **Uniform point sampling**
   The extracted contours are sampled uniformly along arc length to generate a set of target points equal to the number of drones.

These sampled points define the initial spatial configuration of the swarm for video tracking.

It is important to emphasize that edge detection is not used during dynamic tracking. Recomputing edges at every frame would introduce contour flickering and temporal inconsistency, degrading shape preservation without improving motion tracking. Once the initial shape is established:

- The swarm motion is driven entirely by optical flow
- Shape preservation is enforced through rigid-body velocity estimation
- No further image segmentation or edge extraction is performed

Thus, edge detection serves only as a geometric initialization step and does not influence the time evolution of the system.

4. Extracting Motion from Video

4.1 Optical Flow as a Velocity Field

A video provides frames $I(t)$. Optical flow estimates a dense motion field:

$$\mathbf{u}(p,t) = \left(u_x(p,t), u_y(p,t)\right)$$

at each pixel p between consecutive frames.

In implementation, the dense flow is computed using Farnebäck optical flow, producing flow in pixels per frame.

4.2 Mapping Flow to World Units

The simulation uses a world coordinate system $[0, W] \times [0, H]$. Pixel flow must be converted to world velocity:

- scaling factors:

$$s_x = \frac{W_{world}}{W_{img}}, \qquad s_y = \frac{H_{world}}{H_{img}}$$

- convert to "per second" using FPS:

$$V_x = u_x \cdot s_x \cdot FPS, \qquad V_y = -u_y \cdot s_y \cdot FPS$$

(the minus sign flips the image y-axis to match world coordinates).

This ensures the extracted motion has correct magnitude and direction in the simulation space.

4.3 Robustness Enhancement: Flow Smoothing with Strong-Flow Mask

Dense optical flow can be noisy, and objects can "lose lock" when:

- texture is low (blank regions),
- motion blur occurs,
- the object is small.

To reduce instability, the flow field is Gaussian-blurred to spread velocity into nearby pixels:

- blurred flow helps drones slightly off the object still feel the motion ("wake effect"),
- but strong motion regions should remain sharp.

Therefore, a magnitude mask is used:

- if $||\mathbf{u}|| > \tau$, keep original flow (high confidence),
- else use smoothed flow.

This produces a field that is stable enough for control while still respecting object motion.

5. Shape Preservation via Rigid Motion Estimation

5.1 Why Rigid Motion?

If each drone directly follows local optical flow, the swarm can shear, stretch, and become visually incoherent. The assignment requires shape preservation, meaning the swarm should move approximately as a rigid body (translation + rotation).

So, the core idea is:

Use optical flow to estimate a *single rigid motion* (translation + rotation) of the swarm, then drive all drones with that rigid motion.

5.2 Rigid Motion Decomposition

Let the current drone positions be x_i. Define the centroid:

$$c = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad r_i = x_i - c.$$

A 2D rigid velocity model is:

$$v_i^{\text{rigid}} = u + \omega J r_i$$

where:

- $u \in \mathbb{R}^2$ is translation velocity,
- $\omega \in \mathbb{R}$ is angular velocity,
- $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ rotates a vector by 90°,
  so Jr_i is tangent direction for rotation.

5.3 Estimating ω and u from Flow Samples

The flow field gives desired velocities at drone positions:

$$v_i^{flow} \approx V(x_i, t).$$

We fit the rigid model $u + \omega\, Jr_i$ to these samples. A stable closed-form estimate:

1. Estimate angular velocity by projection:

$$\omega = \frac{\sum_i (Jr_i) \cdot v_i^{flow}}{\sum_i |Jr_i|^2 + \varepsilon}.$$

2. Estimate translation as the mean residual:

$$u = \frac{1}{N} \sum_i \left(v_i^{flow} - \omega J r_i \right).$$

Then:

$$v_i^{rigid} = u + \omega J r_i$$

and finally saturate it to respect v_{max}.

This method produces a single coherent motion shared by the whole swarm, which is exactly what "shape preservation" requires.

(Note: In the code I compute omega using a similar projection formula and compute u from the mean residual. That is a practical least-squares rigid fit.)

5.4 Controller Using Velocity Tracking (IVP-VT)

The swarm does not instantly set $v_i = v_i^{rigid}$. that would be jerky. Instead, it uses the IVP-VT acceleration law:

$$\dot{v}_i = \frac{1}{m} \left[k_v \left(v_i^{rigid} - v_i \right) + f_{rep} - k_d v_i \right].$$

This acts like a first-order velocity servo:

- fast response controlled by k_v,
- damping controlled by k_d,
- safety controlled by repulsion f_{rep}.

The position derivative is set to the rigid velocity field (kinematic enforcement), making the shape move as a rigid object, while the velocity dynamics stabilize the motion and handle collisions.

Although the velocity state v_i follows IVP-VT dynamics, the kinematic update uses the rigid velocity v_i^{rigid} to strictly enforce shape preservation, while v_i acts as a filtered control variable.

6. Synchronizing Video Time with Simulation Time

6.1 Two Time Scales

- Simulation integrates with step DT (e.g., 0.02s)
- Video provides motion every 1/FPS seconds:

$$\Delta t_{\text{video}} = \frac{1}{30} \approx 0.0333 \text{ s}.$$

A **time accumulator** is used:

- advance physics every DT,
- only read the next video frame when accumulated time exceeds Δt_{video}

This ensures:

- reproducible sync
- no double-counting of flow frames
- stable motion when $DT \neq \Delta t_{\text{video}}$

6.2 Time-Scale Compensation

Because the flow is scaled to "per second", while the simulation applies DT, the rigid velocity is optionally adjusted by:

$$\text{time\_scale} = \frac{\Delta t_{\text{video}}}{DT}$$

so, motion magnitude matches the rate at which new flow information arrives.

This avoids the common bug where the swarm "overreacts" when the simulation step is smaller than the video frame step.

7. Algorithm Summary

1. **Load state** from Sub-problem 2 checkpoint (reproducible initial condition).
2. **Compute first-frame edge mask** and sample points to form the initial "object shape" configuration.
3. **Morph** from greeting to object shape using Sub-problem 1/2 static model for a few seconds (smooth transition).
4. For each simulation step:
   - update flow field when video time advances,
   - sample flow at drone positions,
   - estimate rigid motion (u, ω),
   - compute rigid velocities v^{rigid},
   - integrate IVP-VT system with RK4,
   - record trajectories.

8. Numerical Method and Justification

8.1 RK4 Integration

The ODE system is solved using **4th-order Runge–Kutta (RK4)**:

$$y_{n+1} = y_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

- high accuracy,
- stable for smooth force fields,
- suitable for moderate stiffness due to damping and saturation.

8.2 Stability Tools Used

To keep the system stable under noisy real video flow:

- velocity saturation v_{max},
- damping k_d,
- repulsion clipping,
- flow smoothing + strong-flow mask,
- bounded simulation duration cap.

Together, these prevent divergence and "exploding drones".

9. Test Cases and Observations

To evaluate the proposed dynamic tracking and shape preservation framework, four videos were tested. All videos were recorded with a static camera. Except for the final case, the scenes were recorded be me, therefore, under non-ideal, realistic conditions (hand-held motion paths, imperfect lighting, and non-uniform trajectories), providing a meaningful robustness assessment.

9.1 Successful Cases

Case 1: Ring Translating (from left to right and slightly down)

*Please check input video in inputs folder, with name **my_ring_translation.mp4***

Please check output video in visualizations *folder*, with name **drone_show_my_ring_translation_p3.mp4**

In the first test, a ring-shaped object moves from left to right with a slight downward component. Result shows, that the swarm accurately transitions to the ring shape and follows the object's trajectory smoothly.

- The ring has clear edges and sufficient texture for reliable optical flow.
- Motion is predominantly translational and slow enough to avoid blur.
- The rigid-body approximation holds well.

As a result, the estimated rigid velocity closely matches the true object motion, leading to stable and accurate swarm tracking.

Case 2: Cross Translating and Rotating

*Please check input video in inputs folder, with name **my_rotation.mp4***

Please check output video in visualizations *folder*, with name **drone_show_my_rotation_p3.mp4**

The second test involves a cross-shaped object undergoing simultaneous translation and rotation. The swarm successfully reproduces both the translational motion and the rotational behavior while maintaining shape coherence.

- The cross provides strong directional features, improving flow stability.
- The motion consists of rigid translation plus rotation, exactly matching the assumed motion model.

This case demonstrates that the system correctly captures and reproduces combined rigid motions.

9.2 Unsuccessful test cases

Case 3: Ring Rotating Out of Plane (Perspective Deformation)

Please check input video in inputs folder, with name ***my_ring_deform.mp4***

Please check output video in visualizations *folder*, with name **drone_show_my_ring_deform_p3.mp4**

In this test, the ring rotates in such a way that, from the camera's perspective, it becomes partially degenerate and appears as a line at certain angles. In the resulting simulation, however, the swarm maintains a ring shape throughout the motion.

It happens because.

- Edge detection is used only for initialization, so transient shape distortions do not alter the swarm geometry.
- The rigid motion estimator extracts a consistent global motion from optical flow without re-estimating shape.

This behavior confirms that shape preservation is enforced intentionally and is robust to perspective-induced visual distortions.

Case 4: Two Balls Moving and Colliding

Please check input video in inputs folder, with name ***two_balls.mp4***

Please check output video in visualizations *folder*, with name **drone_show_two_balls_p3.mp4**

The final test features two balls moving toward each other and colliding. In the resulting simulation, the swarm exhibits little to no motion.

This behavior is expected and consistent with both the model assumptions and the assignment specification, which explicitly refers to tracking a *single moving object*, not multiple independent objects.

In this scenario:

- Optical flow captures two distinct and conflicting motion patterns.
- The rigid-motion estimator attempts to fit a *single* translation and rotation to incompatible flow vectors, causing the estimated velocity to average toward zero.
- No object segmentation or multi-object tracking is implemented, as this was outside the scope of the assignment.

Consequently, the swarm cannot follow either ball in a meaningful way. This result does not indicate a failure of the implementation, but rather a deliberate limitation imposed by the single-object rigid-motion assumption.

10. Conclusion

This project established a numerical framework for video-driven drone swarm control with shape preservation. The swarm transitions from a static configuration to mimic a detected object's shape, subsequently tracking its motion over time. To maintain geometric coherence, dense optical flow is reduced to a rigid-body velocity model (comprising translation and rotation) and stabilized via damping, repulsion, and velocity saturation. The governing equations are formulated as an initial value problem with velocity tracking (IVP-VT) and solved using a fourth-order Runge–Kutta method, with precise time synchronization ensuring physical consistency across varying frame rates.

Experimental results demonstrate that the approach effectively tracks single rigid objects, even under non-ideal conditions. Identified failure cases, such as perspective-induced deformations or multi-object motion, align with the model's underlying assumptions rather than implementation errors. Ultimately, this stable and interpretable pipeline successfully integrates computer vision and control theory, providing a robust foundation for future extensions into non-rigid shape adaptation and multi-object tracking.

11. AI Usage

AI tools (chatgpt.com grok.com) were used during the development of this project as an auxiliary aid, not as a replacement for independent work or understanding.

Their usage was limited to:

- Improving code readability through comments, refactoring suggestions, and documentation.
- Discussing reasonable parameter ranges to support numerical stability and smooth visualization.
- Clarifying conceptual aspects such as IVP formulation, rigid-body motion estimation from optical flow, and numerical integration choices.
- Polishing the report's language, structure, and mathematical notation.
- Generate the video of 2 opposite moving balls.

All algorithmic decisions, mathematical models, numerical methods, and final implementations were designed, validated, and executed independently by me. No AI agent was used to automatically generate the final solution, codebase, or experimental results.